



**SAPIENZA**  
UNIVERSITÀ DI ROMA

# Residual Learning for MPC-Based Control of the TITA Biped Wheeled-Legged Robot

Department of Computer, Control and Management Engineering  
Artificial Intelligence and Robotics

**Jacopo Tedeschi**

ID number 1882789

Advisor

Prof. Giuseppe Oriolo

Co-Advisors

Prof. Leonardo Lanari

Dott. Nicola Scianca

Academic Year 2025/2026

Thesis not yet defended

---

**Residual Learning for MPC-Based Control of the TITA Biped Wheeled-Legged Robot**

Master Degree. Sapienza University of Rome

© 2025/2026 Jacopo Tedeschi. All rights reserved

This thesis has been typeset by L<sup>A</sup>T<sub>E</sub>X and the Sapthesis class.

Author's email: [tedeschi.1882789@studenti.uniroma1.it](mailto:tedeschi.1882789@studenti.uniroma1.it)

## Abstract

Model-based control and Reinforcement Learning address different limitations in robotic locomotion. Model Predictive Control (MPC) uses an explicit dynamics model to optimize future motion and handle physical constraints, but its performance can degrade when the model does not capture the full behavior of the system. Reinforcement Learning (RL) can learn behaviors directly through interaction, making it suitable for compensating for effects that are difficult to model. However, policies trained from scratch often require extensive reward design and provide little insight into the resulting control strategy. This thesis investigates a hybrid approach in which a learned policy augments a nominal MPC controller instead of replacing it. The model-based component provides the main control action, while the policy learns corrections aimed at improving closed-loop performance. The framework is implemented in JAX using MPX to parallelize the MPC computations required during training and is evaluated in the MuJoCo physics engine. Its performance is assessed against both model-based and learning-based baselines, with particular attention to tracking accuracy, robustness, and generalization across different commands and operating conditions.



# Contents



# Chapter 1

## Introduction

Legged robots can traverse environments that remain difficult or inaccessible to conventional wheeled platforms. Their ability to establish and break contacts with the ground allows them to negotiate obstacles, uneven surfaces, and discontinuous terrain. This mobility, however, comes at the cost of a substantially more complex control problem. Locomotion requires the coordinated motion of several joints while the robot maintains balance, regulates contact forces, and tracks the commanded velocity. These objectives must be satisfied despite impacts, friction uncertainty, actuator limitations, and changes in the contact configuration.

Wheeled-legged robots combine articulated limbs with driven wheels and occupy an intermediate position between legged and wheeled systems. On regular surfaces, rolling motion provides greater efficiency and speed than repeated stepping. When the terrain becomes irregular, the legs can change the height and orientation of the body, redistribute the contact forces, and step over obstacles that cannot be crossed through rolling alone. This combination extends the range of possible motions, but also introduces further control challenges. The controller must account for the floating-base dynamics, the unilateral nature of contacts, the rolling constraints of the wheels, and the interaction between body motion and wheel traction. At higher velocities, changes in load distribution and imperfect contact modelling can further reduce stability [?].

Model Predictive Control (MPC) has become one of the main approaches to locomotion control because it expresses the task as an Optimal Control Problem. At each control update, MPC predicts the evolution of the robot over a finite horizon and computes a sequence of inputs that minimizes a cost function while satisfying a model of the dynamics and a set of constraints. Only the first input of the optimized sequence is applied. The optimization is then repeated from the updated state. This structure allows the controller to consider future motion rather than reacting only to the current tracking error. It also provides a direct way to encode objectives such as velocity tracking, body-height regulation, force distribution, friction limits, and actuator bounds.

The explicit formulation of the prediction model, objective function, and constraints makes MPC relatively transparent. Predicted trajectories and contact forces can be inspected, and individual weights can be adjusted according to their physical meaning. This is particularly useful in legged robotics, where controller failures may

arise from several interacting sources. The internal solution provides information that can help distinguish an infeasible motion request from poor state estimation, incorrect contact assumptions, or insufficient control authority.

This structure does not remove the difficulties associated with modelling locomotion. A full rigid-body model describes the robot more accurately, but its use over a sufficiently long prediction horizon may be too computationally expensive for real-time control. Reduced-order models are therefore commonly adopted. Examples include centroidal models, Single Rigid Body Dynamics models, and simplified inverted-pendulum representations. These formulations retain the quantities that dominate the global motion of the robot while omitting part of the articulated dynamics. The reduction makes online optimization practical, but creates a discrepancy between predicted and actual behaviour. Joint dynamics, actuator response, structural compliance, impacts, and complex wheel–terrain or foot–terrain interactions may only be represented approximately.

When a reduced model is used, a Whole-Body Controller (WBC) can be introduced between the MPC and the actuators. Its purpose is to map the motion quantities and contact forces planned by MPC to commands that are consistent with the complete kinematics and dynamics of the robot. A WBC can be formulated as a Quadratic Programming (QP) problem in which generalized accelerations, contact forces, or joint torques are selected to track the reduced-order plan. At the same time, the QP can account for additional requirements that are absent from the simplified MPC model, including the floating-base equations of motion, friction-cone conditions, joint limits, contact constraints, swing-foot tracking, body-orientation regulation, and posture regularization. The resulting hierarchy separates long-horizon motion planning from instantaneous whole-body control. It does not eliminate model mismatch, but it allows several physical constraints to be enforced without introducing the complete multibody system into every node of the MPC horizon.

The performance of the resulting MPC-WBC architecture still depends on the validity of its models and contact assumptions. Errors in mass and inertia parameters, terrain height, friction, actuation delay, or contact timing can produce persistent tracking errors and, in more severe cases, instability. Some effects can be addressed by increasing model fidelity or adding constraints and cost terms. This solution is not always practical. A more detailed formulation increases the computational burden, and some phenomena are difficult to describe through a compact differentiable model. The controller therefore faces a persistent compromise between physical accuracy and the ability to solve the optimization problem at the required frequency. Most established MPC solvers used in robotics are designed primarily for CPU execution. Although these implementations can achieve the update rates required to control a single robot, their computational cost becomes a bottleneck when the controller must be evaluated across thousands of parallel simulation environments. This requirement is particularly relevant when MPC is included directly in a Reinforcement Learning training loop. Recent work has therefore focused on reformulating optimal-control algorithms to exploit GPU parallelism, allowing multiple MPC problems to be solved concurrently alongside the learning process. MPX follows this direction by providing a JAX-based, GPU-accelerated MPC framework for legged robots [?]. Its solver exploits temporal and state-space parallelism through associa-

tive scan operations, enabling the model-based controller to remain active during large-scale policy training rather than being used only to generate offline trajectories or reference motions. Its solver exploits temporal and state-space parallelism through associative scan operations, reducing the dependence of computation time on the prediction-horizon length and on the dimensions of the state and control spaces. Besides supporting real-time control, this formulation makes it possible to evaluate many MPC instances concurrently during learning. Model-based control can therefore remain inside the simulation loop rather than being used only to generate an offline dataset or a reference trajectory.

Reinforcement Learning (RL) offers a different approach to the same control problem. Rather than requiring a complete analytical description of the dynamics, an RL policy learns a mapping from observations to actions through interaction with an environment. The policy is optimized to maximize the expected cumulative reward, which can include velocity tracking, stability, energy consumption, action smoothness, and other task-related objectives. Training under variations in the robot model, terrain, observations, and external disturbances can produce behaviours that are less dependent on a single nominal model. RL can also account implicitly for nonlinear contact effects and objectives that would be cumbersome to include in an online optimization problem.

These advantages are accompanied by important limitations. A learned policy does not normally enforce physical constraints in the explicit form used by MPC. Its behaviour depends strongly on the training distribution, reward definition, action representation, and simulation model. Good performance inside the training domain does not guarantee predictable behaviour in states that were rarely or never encountered. Reward design can also require considerable tuning, since a policy may exploit unintended aspects of the simulator or trade one objective against another in an undesirable way. Finally, the internal reasoning of a neural policy is difficult to inspect. Diagnosing a failure is therefore less direct than examining the state, constraints, and cost terms of a model-based optimizer.

MPC and RL have complementary strengths. MPC supplies a physically grounded nominal behaviour, explicit predictions, and structured constraint handling. RL provides a mechanism for learning corrections from experience when the nominal model is incomplete. This complementarity has motivated several combined architectures. A policy may adjust parameters of the optimization problem, generate reference trajectories or contact schedules, imitate trajectories produced by an MPC expert, or act in parallel with the model-based controller [?, ?]. Each choice determines how authority is divided between the analytical controller and the learned component.

This thesis focuses on residual learning, in which the model-based controller remains active and the policy produces a correction to its nominal command. The main locomotion task is therefore not learned from an unstructured initial policy. MPC and Whole-Body Control provide the nominal motion and the corresponding actuator torques, while RL learns only the additional contribution required to improve the closed-loop response. The final command can be expressed in the general form

$$\tau_{\text{cmd}} = \tau_{\text{nom}} + \tau_{\text{res}},$$

where  $\tau_{\text{nom}}$  is generated by the model-based control branch and  $\tau_{\text{res}}$  is obtained from the policy output through a low-level feedback law. The nominal controller provides a strong control prior and keeps the learning process close to a physically meaningful locomotion strategy. The policy is then free to compensate for systematic tracking errors, simplified dynamics, uncertain contacts, and other effects that are not captured adequately by the nominal formulation. Similar torque-level residual architectures have shown that an MPC prior can improve training efficiency and extend the range of trackable locomotion commands [?].

The residual structure also provides a natural experimental baseline. When the learned contribution is bypassed, the robot is controlled by the original MPC and WBC architecture. When it is enabled, the nominal controller and experimental conditions remain unchanged, while the policy adds its correction. The effect of learning can therefore be evaluated without comparing two entirely different locomotion pipelines. This distinction must be implemented carefully: setting the policy action to zero does not necessarily remove every torque produced by a residual feedback controller. A genuine baseline requires the residual branch to be excluded from the actuator command.

The work presented in this thesis studies this approach on two robotic systems in the MuJoCo physics engine [?]. MuJoCo provides rigid-body dynamics, contact simulation, and an interface suitable for parallel learning and control experiments. Both the nominal controllers and the learned policies are evaluated within the same simulated dynamics, allowing their behaviour to be compared under consistent initial conditions, velocity commands, disturbances, and model variations.

The first platform is TITA, a wheeled-legged robot with two articulated legs and two actuated wheels. Its nominal MPC is based on the Double Force Constrained Inverted Pendulum model, denoted as DFCIP. This reduced-order representation describes the quantities that dominate the motion of the robot, including its centre-of-mass position and velocity, heading, forward velocity, yaw rate, and wheel contact geometry. From the measured robot state and the commanded forward and angular velocities, the MPC plans the desired forward, vertical, and angular accelerations together with the ground reaction forces at the two wheels.

These accelerations and ground reaction forces constitute the reduced-order references supplied by the MPC to the Whole-Body Controller. The WBC formulates an instantaneous Quadratic Program over the generalized accelerations and wheel contact forces. Its objective includes tracking tasks for the centre of mass, the left and right wheels, the base orientation, and the joint posture. The optimization also accounts for the floating-base equations of motion, friction constraints, and predicted joint limits. Once the QP has been solved, inverse dynamics maps the optimized generalized accelerations and contact forces to the nominal torques of the eight actuators. The WBC therefore connects the accelerations and ground reaction forces produced by the DFCIP-MPC to the complete multibody dynamics of TITA. For TITA, the residual policy observes the robot state together with selected outputs of the model-based controller. These include the accelerations and ground reaction forces planned by MPC and the nominal torques computed by WBC. The policy therefore has access not only to the measured motion but also to the action intended by the nominal controller. Its outputs modify the desired positions of the leg joints and the desired velocities of the wheels. Joint-level feedback converts

these offsets into residual torques, which are added to the nominal WBC torques before the complete command is applied to the actuators.

The second platform is Lite3, a quadruped robot controlled through an SRBD-based MPC and a Whole-Body Controller. The MPC receives planar velocity commands and predicts the motion of the robot base while optimizing the ground reaction forces of the four feet over a finite horizon. A predefined gait generator determines the stance and swing phases and constructs the corresponding foot references. The WBC uses the first optimized set of contact forces for stance control and tracks the planned foot trajectories during swing, producing the nominal joint torques for all twelve actuators.

The Lite3 residual policy generates bounded offsets from a nominal joint posture. A joint-space proportional-derivative controller converts these offsets into residual torques, which are added to the output of the MPC-WBC branch. The policy thus modifies the resulting whole-body behaviour without replacing the planner or directly altering the MPC optimization. The Lite3 study also permits comparison with an end-to-end RL controller, making it possible to examine the trade-off between a purely model-based solution, a residual architecture, and a policy that learns locomotion without the same online model-based prior.

Both residual policies are trained using Proximal Policy Optimization [?], an on-policy policy-gradient algorithm that alternates between collecting trajectories and performing multiple optimization epochs on a clipped surrogate objective. Training is performed in parallel MuJoCo environments, with the model-based controller active inside each environment. The reward promotes tracking of the commanded linear and angular velocities while penalizing unstable body motion, excessive actuator effort, rapid changes in the learned action, and episode termination. During training, the critic may use privileged simulation quantities that are not required by the deployed actor. Observation noise and disturbances can also be introduced to reduce dependence on ideal measurements and to test whether the learned correction remains effective away from nominal conditions.

The objective of this thesis is to determine whether residual learning can improve the tracking accuracy and robustness of an MPC-based locomotion controller without discarding its physical structure. The investigation considers individual and combined velocity commands, stability limits, control effort, uneven terrain, disturbances, observation noise, and variations in model parameters. The comparison is not limited to the final tracking error. It also examines whether improvements are obtained through bounded and sufficiently smooth corrections, whether the nominal behaviour is preserved when no correction is needed, and whether the learned component introduces systematic biases or undesirable interactions with the original controller.

The main contribution is the development and evaluation of two residual MPC architectures for robots with different morphologies and contact behaviours. The first combines a DFCIP-based MPC, a constrained WBC, and joint-level residual control on a two-wheeled legged platform. The second applies the same general principle to a quadruped using SRBD-MPC, gait scheduling, and stance-swing whole-body control. In both cases, MPC remains active during policy training and evaluation. This allows the policy to learn within the same closed-loop architecture used at evaluation and provides a direct comparison against the unmodified model-based

controller. The study further analyses the practical consequences of residual torque addition, including action representation, controller update rates, baseline isolation, and the interaction between the learned and nominal commands.

The remainder of the thesis develops this investigation from its theoretical foundations to its experimental evaluation. The next chapter reviews model-based, learning-based, and hybrid approaches to legged locomotion. The robot models and the reduced-order representations used by the controllers are then introduced, followed by the formulation of MPC and Whole-Body Control. The reinforcement-learning problem, policy observations, actions, and training procedure are subsequently described. The complete residual architectures for TITA and Lite3 are then presented, including the exchange of information between the model-based and learned components. Finally, the controllers are evaluated in simulation, the results and limitations are discussed, and the main conclusions and possible directions for future work are reported.

## Chapter 2

# Background and Related Work

This chapter reviews the main approaches to the control of legged and wheeled-legged robots, with particular attention to the combination of model-based control and reinforcement learning. The discussion begins with the characteristics of these robotic platforms and the challenges introduced by dynamic locomotion, intermittent contacts, and underactuated floating-base dynamics.

Model-based methods are then examined through reduced-order models, Model Predictive Control, and Whole-Body Control. These techniques provide structured and interpretable control solutions, but their performance depends on model accuracy and is constrained by the computational cost of online optimization. Learning-based locomotion offers a complementary approach, allowing policies to adapt to complex dynamics and uncertain interactions through experience.

The chapter subsequently considers hybrid architectures that combine MPC and Reinforcement Learning. Particular attention is given to hierarchical, residual, and adaptive formulations, which assign different roles to the learned component and the nominal controller. Finally, recent developments in GPU-accelerated optimal control are discussed, since they make it possible to execute many MPC instances in parallel and train reinforcement-learning policies with the optimizer directly in the control loop.

### 2.1 Related Works

Optimization-based control has become a standard approach to dynamic legged locomotion. It formulates motion generation as an Optimal Control Problem (OCP) in which the robot dynamics, contact forces, physical constraints, and task objectives are considered explicitly. Solving the complete problem online is difficult because legged robots have many degrees of freedom and interact with the environment through intermittent and unilateral contacts [?]. Existing controllers therefore differ in the dynamic model they employ, the way they represent contacts, and the division of tasks between predictive planning and low-level control.

Reduced-order models make online optimization tractable by retaining the variables that dominate balance and locomotion while neglecting part of the articulated dynamics. The convex MPC developed for the MIT Cheetah 3 represents the trunk as a single rigid body and optimizes the ground reaction forces over a finite hori-

zon [?]. Whole-body nonlinear MPC instead includes the articulated dynamics and optimizes robot motion and contact interactions within a single problem [?]. This richer representation reduces the separation between planning and control but results in a more demanding optimization. Other work has addressed the numerical treatment of three-dimensional orientation [?] and adapted solver accuracy to the time constraints of whole-body locomotion control [?].

Wheeled-legged robots introduce rolling contacts and additional actuated degrees of freedom. Early experiments on ANYmal integrated wheel motion and non-holonomic rolling constraints into a whole-body planning and control framework [?]. Online trajectory optimization was later used to produce hybrid walking-driving motions on rough terrain [?]. A subsequent whole-body MPC formulation jointly optimized torso motion, wheel motion, and contact forces while adapting the contact sequence online [?]. These works showed that the wheels cannot be treated as an independent drivetrain. Their motion must be coordinated with the leg configuration, body dynamics, and contact forces.

Research on wheeled-legged systems has mainly addressed terrain traversal and the transition between rolling and stepping. At higher speeds, vehicle-like effects become increasingly relevant. Acceleration and cornering redistribute the normal loads among the wheels, reducing the available contact margin and increasing the risk of slip or rollover. A recent controller for the Unitree Go2-W combined an MPC planner for active roll regulation with a learned low-level policy. The legs were used as an active suspension system to lean the body into turns and reduce lateral load transfer during physical racing experiments [?].

Reinforcement Learning provides a different approach to the uncertainties and contact interactions involved in locomotion. Policies trained in simulation have been transferred to physical quadrupeds for agile movements and locomotion over challenging terrain [?, ?]. Perceptive policies have also demonstrated locomotion in natural environments using proprioceptive and exteroceptive information [?]. These methods can learn behaviours that are difficult to describe through an analytical model, but their performance depends on the training distribution, reward formulation, observation design, and simulation accuracy.

The computational structure of RL has encouraged massively parallel simulation. Thousands of robots can be simulated simultaneously on a GPU, producing large batches of experience for policy optimization. This approach has substantially reduced the time required to train locomotion policies and supports curriculum learning across many parallel environments [?]. A purely model-free policy, however, does not explicitly encode the physical constraints or the structure available to an optimization-based controller. This limitation has motivated methods that combine learning and model-based control.

One family of hybrid methods uses model-based optimization to generate references or guidance for a learned policy. Deep Tracking Control computes reference motions through trajectory optimization and trains an RL policy to track them, while retaining the planner during deployment [?]. A related approach uses a reduced-order model to predict desired footholds and lets RL realize those footholds without tracking a complete reference trajectory [?]. In both cases, the model guides the learned behaviour, while the policy remains responsible for producing the low-level control action.

Hierarchical methods assign planning and execution to different components. The policy may select footholds, contact timings, gait parameters, or navigation commands, while MPC computes a dynamically consistent motion. In [?], a high-level RL policy selects acyclic contact patterns and navigation commands for a low-level MPC. This removes the combinatorial search over contact sequences from the continuous optimization and allows the same architecture to address legged and hybrid wheeled-legged locomotion.

A separate family uses learning to modify the model or parameters employed by the optimizer. Learned corrections have been introduced to reduce the discrepancy between reduced-order predictions and the full-order dynamics of a quadruped [?]. Modular residual methods apply learned corrections to individual parts of a model-based controller, including its dynamics model and footstep planner [?]. In the adaptive architecture presented in [?], RL modifies the predictive dynamics, swing-foot parameters, and gait frequency used by MPC. The learned action therefore changes the optimization problem rather than directly correcting the actuator command.

Residual policy learning adopts a more direct decomposition. A conventional controller supplies a nominal action, while RL learns only the correction needed to improve the resulting behaviour. This principle was introduced for contact-rich manipulation tasks, where a feedback controller captured the known structure and a residual policy compensated for effects that were difficult to model [?]. In locomotion, a residual can be applied to the dynamic model, a planned reference, a joint setpoint, or a torque command. These alternatives differ in the authority assigned to the policy and in how much of the nominal controller remains responsible for physical consistency.

The work most closely related to this thesis is Residual MPC, which executes an MPC controller and a residual policy concurrently and combines their outputs at the joint-control level [?]. The MPC provides a structured control prior, while the policy learns corrections for modelling errors and behaviours that are difficult to encode in the optimization. The paper compares different residual action spaces and blending strategies, showing that the interface between the two controllers affects both learning and final locomotion behaviour. Unlike approaches based on offline demonstrations, MPC remains active during training and deployment.

This integration creates a substantial computational requirement. An MPC problem must be solved for every simulated robot at each control update. Optimal-control solvers for legged locomotion have traditionally been designed for sequential execution on CPUs, whereas modern RL environments simulate thousands of agents in parallel on GPUs. The resulting separation between CPU optimization and GPU training limits the number of MPC instances that can be kept inside the learning loop [?, ?].

CusADi addressed this problem by translating symbolic expressions into GPU-compatible operations and evaluating large batches of approximate optimal-control problems in parallel [?]. ReLU-QP followed a different direction by reformulating an ADMM-based quadratic-programming solver using operations efficiently supported by machine-learning frameworks and GPUs [?]. MPX extends GPU acceleration to a primal-dual iLQR formulation implemented in JAX. Associative scans parallelize computations along the prediction horizon, while batching supports many indepen-

dent MPC instances on the same device [?]. This enables MPC to remain directly inside a massively parallel RL training loop.

The literature therefore combines model-based control and learning at several different levels. The model may generate demonstrations, supply reference motions, determine contact sequences, define the dynamics used for prediction, or produce the nominal actuator command. In turn, the learned policy may track a model-based plan, select high-level decisions, adapt the optimization model, or provide a residual correction. This thesis focuses on the last relation: the model-based controller supplies the nominal behaviour, while RL learns a correction through interaction with the simulated system. The exact interface between the two components depends on the robot platform and is described in the architecture chapter.

## 2.2 Legged and Wheeled-Legged Locomotion

Legged robots move through the environment by establishing discrete contacts between their limbs and the surrounding terrain. Unlike conventional wheeled vehicles, they do not require a continuous and approximately flat path. Each foot can be placed on a separate support surface, allowing the robot to cross steps, gaps, debris, and other irregular terrain. This mobility makes legged systems suitable for environments in which fixed infrastructure is unavailable or ordinary vehicles cannot operate [?].

This capability introduces a demanding control problem. A legged robot is a floating-base mechanical system whose body is not directly actuated. Its motion must be generated indirectly through the forces exchanged at the active contacts. These contacts change during locomotion and are governed by unilateral and friction constraints: a foot can push against the terrain but cannot pull on it, while excessive tangential force may cause slipping. The controller must coordinate joint motion, contact timing, and ground reaction forces while maintaining balance and tracking the desired movement [?].

The complexity of locomotion also depends on the selected gait. Periodic gaits, such as walking, trotting, and bounding, define recurring sequences of stance and swing phases. During stance, the contacting limbs support and accelerate the body. During swing, the remaining limbs are repositioned for the next contact. Faster motions reduce the time available for feedback and may include phases with a small support region or no ground contact. Aperiodic locomotion provides more freedom to react to the terrain, but selecting contact timings while planning the continuous motion makes the control problem considerably harder [?, ?].

Model uncertainty further separates the predicted and actual motion. Impacts, actuator limitations, delayed measurements, uncertain friction, and imperfect terrain information cannot always be represented accurately. These effects are particularly important when a controller relies on a reduced-order model, since the neglected articulated dynamics still influence the physical robot. Robust locomotion therefore requires continuous feedback and repeated adaptation of the planned motion.

Wheeled-legged robots extend this mobility by placing wheels at the ends of some or all limbs. On regular terrain, rolling avoids the repeated acceleration and deceleration associated with conventional stepping and can provide faster and more

energy-efficient locomotion [?, ?]. When the terrain becomes discontinuous or unsuitable for rolling, the articulated legs can adjust the body posture, negotiate obstacles, or initiate a stepping motion [?]. The same platform can therefore use rolling for efficient traversal and leg motion when the geometry of the environment requires it.

The mechanical arrangement of the wheels determines the available mobility. Some platforms use steerable wheels, whereas others mount non-steerable wheels at the ends of the legs. In the latter case, each contact can move along the rolling direction but should not slide laterally. The controller must enforce this non-holonomic condition while coordinating wheel velocity, leg configuration, body motion, and contact forces [?, ?]. The wheels increase the set of feasible motions, but also couple locomotion and posture control more tightly than in a purely legged system.

Wheeled-legged locomotion can be divided into rolling, stepping, and hybrid motions. During rolling, the wheels remain in contact with the ground and the legs regulate the posture of the body. During stepping, one or more wheel contacts are lifted and repositioned as ordinary feet. Hybrid locomotion combines these modes within the same manoeuvre, allowing the robot to drive across traversable regions and step over local obstacles [?]. The contact sequence may be prescribed in advance or adapted online according to the state of the robot and its kinematic limits [?].

The addition of wheels does not remove the difficulties associated with legged locomotion. Rolling contacts introduce further constraints, and transitions between driving and stepping modify the support configuration. At higher speeds, acceleration and cornering also redistribute the normal loads between the contacts. This lateral load transfer reduces the available contact margin and can increase the risk of wheel slip or rollover. On skid-steered platforms, yaw is generated through differential wheel torques, further coupling longitudinal motion, lateral dynamics, and body posture [?].

Articulated legs can be used to mitigate these effects. By changing the relative extension of the legs on either side of the body, a wheeled-legged robot can regulate its roll angle and redistribute the contact loads. Physical experiments on the Unitree Go2-W demonstrated this principle by using the legs as an active suspension system and leaning the body into turns [?]. This extends the role of the legs beyond obstacle traversal: their configuration also affects stability and vehicle-like dynamics during fast rolling.

Legged and wheeled-legged robots thus share the need to regulate an underactuated floating body through constrained ground contacts. Wheels improve continuous mobility on suitable terrain, but introduce rolling constraints, additional actuation, and high-speed effects that are absent or less prominent in conventional stepping. These properties motivate controllers that can plan body motion and contact forces online while remaining sufficiently efficient for closed-loop operation.

## **2.3 Model-Based Locomotion Control**

### **2.3.1 Reduced-Order Models**

### **2.3.2 Model Predictive Control**

### **2.3.3 Whole-Body Control**

## **2.4 Learning-Based Locomotion Control**

### **2.4.1 End-to-End Reinforcement Learning**

### **2.4.2 Residual Reinforcement Learning**

## **2.5 Hybrid MPC and Reinforcement Learning Architectures**

### **2.5.1 Hierarchical Architectures**

### **2.5.2 Residual Architectures**

### **2.5.3 Adaptive MPC Architectures**

## **2.6 GPU-Accelerated Optimization and Learning**

## **2.7 Positioning of This Thesis**

## Chapter 3

# Conclusions

This is temporary content.